MICROSERVICIOS · PRIMER PREVIO · UFPS 2026

CodeLens Pro Max

Analizador de código basado en microservicios



Carlos Alejandro Quiñones Martínez

Código: 1151805

Ingeniería de Sistemas · Universidad Francisco de Paula Santander

Análisis del sistema

¿Qué problema resuelve CodeLens Pro Max?

El problema

- Los proyectos acumulan vulnerabilidades (inyección SQL, secretos expuestos, criptografía débil) sin que nadie las vea.
- Dependencias con versiones vulnerables pasan desapercibidas.
- La deuda técnica (complejidad, duplicación) crece sin una medida objetiva.
- Revisar a mano es lento y no dice por dónde empezar.

La propuesta

Una plataforma web que recibe el código, lo analiza y devuelve un panel con nota, hallazgos y plan de acción.

ZIP — **Archivo .zip**
Subida de hasta 200 MB

DIR — **Carpeta local**
El navegador la empaqueta

GIT — **Repositorio Git**
Clonado con JGit



Resultado

- Nota A–F
- Puntajes
- Hallazgos con archivo y línea
- Plan P0–P3

Exporta el reporte a Markdown y HTML imprimible.

Análisis del sistema

Actor, requisitos funcionales y no funcionales

ACTOR

Desarrollador / usuario

- Se registra e inicia sesión
- Importa un proyecto
- Lanza el análisis
- Consulta y descarga reportes
- Ve solo su propio historial

Funcionales

RF1 Registro y login con JWT

RF2 Importar código por .zip, carpeta o Git

RF3 Inventario de archivos y lenguajes

RF4 Detectar vulnerabilidades, secretos y dependencias inseguras

RF5 Medir complejidad, duplicación y deuda técnica

RF6 Generar veredicto, resumen y recomendaciones priorizadas

RF7 Historial por usuario y exportación MD / HTML

No funcionales

Seguridad
Token validado solo en el gateway; X-User-Id no se puede falsificar

Independencia
Una base de datos por servicio, sin tablas compartidas

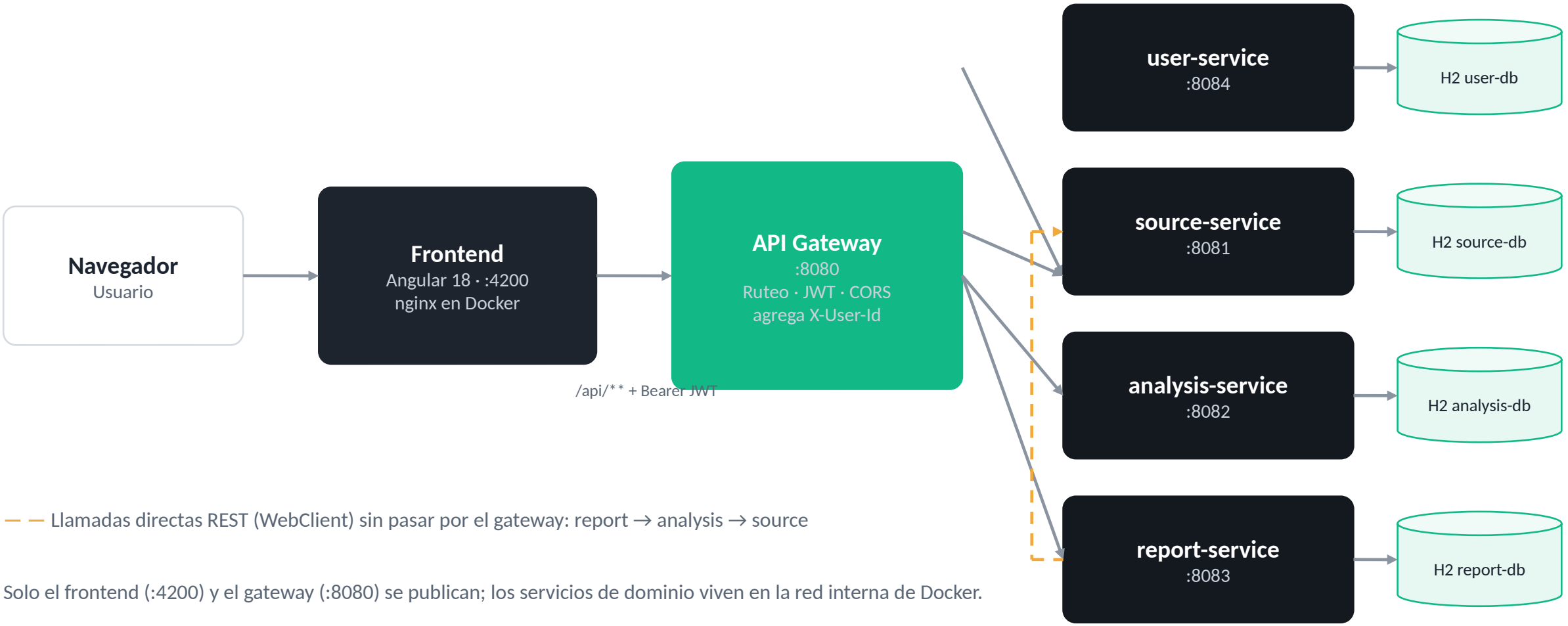
Despliegue
Un Dockerfile por servicio y Docker Compose

Consistencia
Formato de error único en todos los servicios

Observabilidad
/actuator/health en cada servicio

Diseño de la arquitectura

Frontend → API Gateway → microservicios de dominio, cada uno con su base de datos



— — Llamadas directas REST (WebClient) sin pasar por el gateway: report → analysis → source

Solo el frontend (:4200) y el gateway (:8080) se publican; los servicios de dominio viven en la red interna de Docker.

Decisiones de diseño

Patrones aplicados en la arquitectura

1

API Gateway

Spring Cloud Gateway es el único punto de entrada: enruta /auth, /users, /sources, /analyses y /reports, centraliza CORS y valida el JWT.

2

Database per service

Cada servicio tiene su propia base H2 y ningún otro la toca. Los datos se comparten solo por API.

3

Seguridad en el borde

El gateway borra cualquier X-User-Id del cliente y pone el del token. Cada servicio guarda el ownerId y filtra por él.

4

Comunicación síncrona REST

Los servicios de dominio se llaman directo con WebClient: el reporte pide el análisis y el análisis pide los archivos a source.

5

Errores uniformes

Todos responden {timestamp, status, error, message, path} con un GlobalExceptionHandler por servicio.

6

Contenedores independientes

Un Dockerfile por servicio, orquestados con Docker Compose y volúmenes de datos separados.

Microservicios implementados

5 servicios Spring Boot + frontend Angular, todos funcionando de punta a punta

:8080

gateway

Enrutamiento, filtro JWT, CORS e inyección de X-User-Id.

BASE DE DATOS
—

:8084

user-service

Registro, login con BCrypt y emisión de tokens JWT (jjwt). GET /users/me.

BASE DE DATOS
user-db

:8081

source-service

Ingesta: clona Git con JGit, descomprime .zip o carpeta, inventario y detección de lenguaje.

BASE DE DATOS
source-db + workspaces

:8082

analysis-service

Motor de reglas: inyección, secretos, criptografía, dependencias vulnerables, complejidad y duplicación.

BASE DE DATOS
analysis-db

:8083

report-service

Veredicto, resumen ejecutivo y recomendaciones P0–P3 con playbooks. Exporta MD y HTML.

BASE DE DATOS
report-db

Flujo de un análisis

Cada paso lo atiende un microservicio distinto

1 — Autenticarse
user-service
`POST /auth/login`

2 — Importar código
source-service
`POST /sources/git · zip · folder`

3 — Analizar
analysis-service
`POST /analyses`

4 — Generar reporte
report-service
`POST /reports`

5 — Consultar
report-service
`GET /reports · /markdown · /html`

Dentro del analysis-service

49
reglas en 5 catálogos JSON

5
analizadores del motor

P0–P3
prioridad del plan de acción

- **RuleBasedAnalyzer** — inyección, crypto, config, deuda
- **EntropySecretAnalyzer** — secretos y llaves
- **DependencyAnalyzer** — librerías vulnerables
- **CodeHealthAnalyzer** — complejidad y tamaño
- **DuplicationAnalyzer** — código duplicado

Stack y estado actual

Backend	Java 21 · Spring Boot 3.3 · Spring Cloud Gateway 2023
Persistencia	Spring Data JPA · H2 (una por servicio)
Seguridad	JWT (jjwt 0.12) · BCrypt
Ingesta	JGit · Commons Compress
Frontend	Angular 18 standalone + signals
Infraestructura	Docker · Docker Compose · nginx

Estado

- ✓ Los 5 microservicios compilan y corren (scripts y Docker Compose)
- ✓ Flujo completo: login → importar → analizar → reportar
- ✓ Prueba smoke de punta a punta vía gateway
- ✓ 401 sin token, 409 usuario duplicado, aislamiento entre usuarios
- ✓ Frontend con pipeline visual, reporte e historial

¡Gracias! ¿Preguntas?